MIDDLESEX UNIVERSITY

THE BURROUGHS, HENDON, LONDON NW4 4BT

Title: Coursework 2 – Social Networking Website Artwave

Student Name/Surname: Zarif Saleem

Student ID: M01036102

Word Count: 2,291

Table of Contents

Introduction 3
Project Proposal 3
Wireframe 4
Front End of the Website (HTML & CSS) 5
Back End of the Website (JavaScript & MongoDB) 9
Database Design 10
Backend Testing and API Validation 10
Third-Party API and Web Scraping System 18
Evaluation and Reflection 18
Conclusion 19
Usage of AI 19

Introduction

ArtWave is a social networking web application developed for CST2120 Coursework 2. The main aim of the project was to design and implement a web application that allows users to share artwork and interact with other users through a structured platform.

The idea for ArtWave came from existing social media platforms (like deviantart). The platform allows users to upload their artwork, view posts from other artists, and engage with the community through likes, comments, and follows.

The application was developed as a single-page application, meaning the page does not reload when navigating between sections. This improves the user experience and makes the application feel smoother and more modern.

Project Proposal

The goal of ArtWave was to build a fully functional social media website with both front-end and back-end integration. The project was planned to meet all coursework requirements while also being practical and user focused.

<u>Main Objectives</u>

- Allow users to register and log in securely
- Enable users to upload artwork with images
- Implement a follow system between users
- Display a feed based on followed users
- Allow likes and comments on artwork posts
- Include a creative inspiration feature

The platform is targeted at artists, and designers who want a simple space to showcase their work and interact with others and also for people that want to view different art.

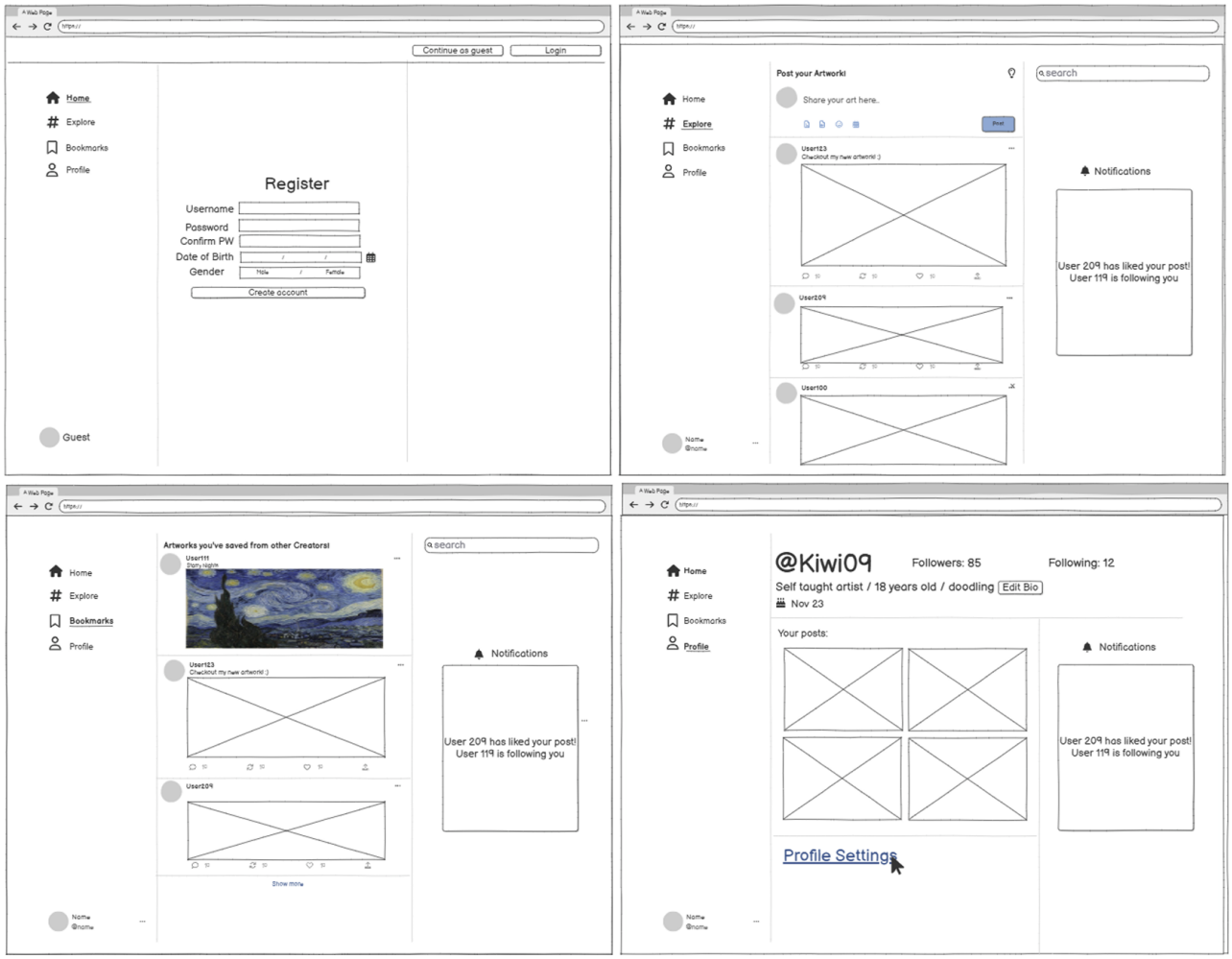
Wireframe

Before starting development, basic wireframes were created to plan the layout and navigation of the website. This helped in understanding how different sections would connect and how users would move through the application.

The main sections planned were:

- Authentication (Login / Register)
- Artwork Feed
- Post Artwork
- Search
- User Profile
- Inspiration Page

The wireframes were simple but effective in guiding the development process and reducing confusion later on.



Front End of the Website (HTML & CSS)

The front end of the ArtWave website was built as a single-page application using one main HTML file, index.html, along with external CSS and JavaScript files. This choice was made to minimize page reloads and create a smoother experience when users move between sections like authentication, posting artwork, viewing feeds, searching, and accessing user profiles.

HTML elements were used throughout the page's structure to make the code easier to read and maintain. Elements such as <header>, <nav>, <main>, <section>, and <footer> clearly distinguish different areas of the website, making it easier for both developers and browsers to understand. Each main feature of the website is housed in its own section, which can be shown or hidden dynamically with JavaScript instead of loading new pages.

The CSS focuses on a clean and modern look that fits an artwork-based social platform. I used a color scheme runs across all pages to keep a strong visual identity. Flexbox and css grid were widely used to create layouts that respond well to various screen sizes. Cards display artworks, user profiles, and feed items, presenting content in a clear and organized way.

Interactive elements like buttons, navigation icons, and cards incorporate hover effects and subtle transitions to enhance usability and give visual feedback to users. Forms for logging in, registering, posting artwork, and searching have spacing and alignment that prioritize readability and user-friendliness. Overall, the front-end design aims to be simple, intuitive, and visually appealing for creative users.

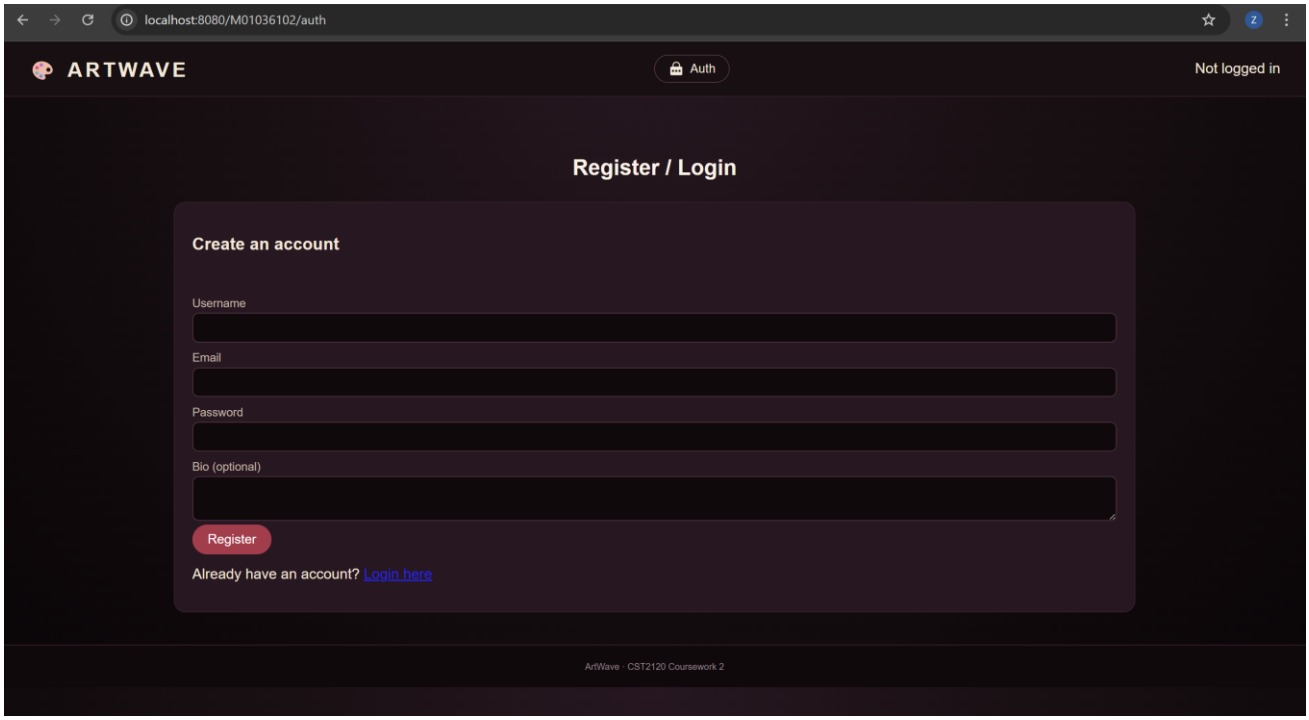


Fig. 1 Register Page (Home)

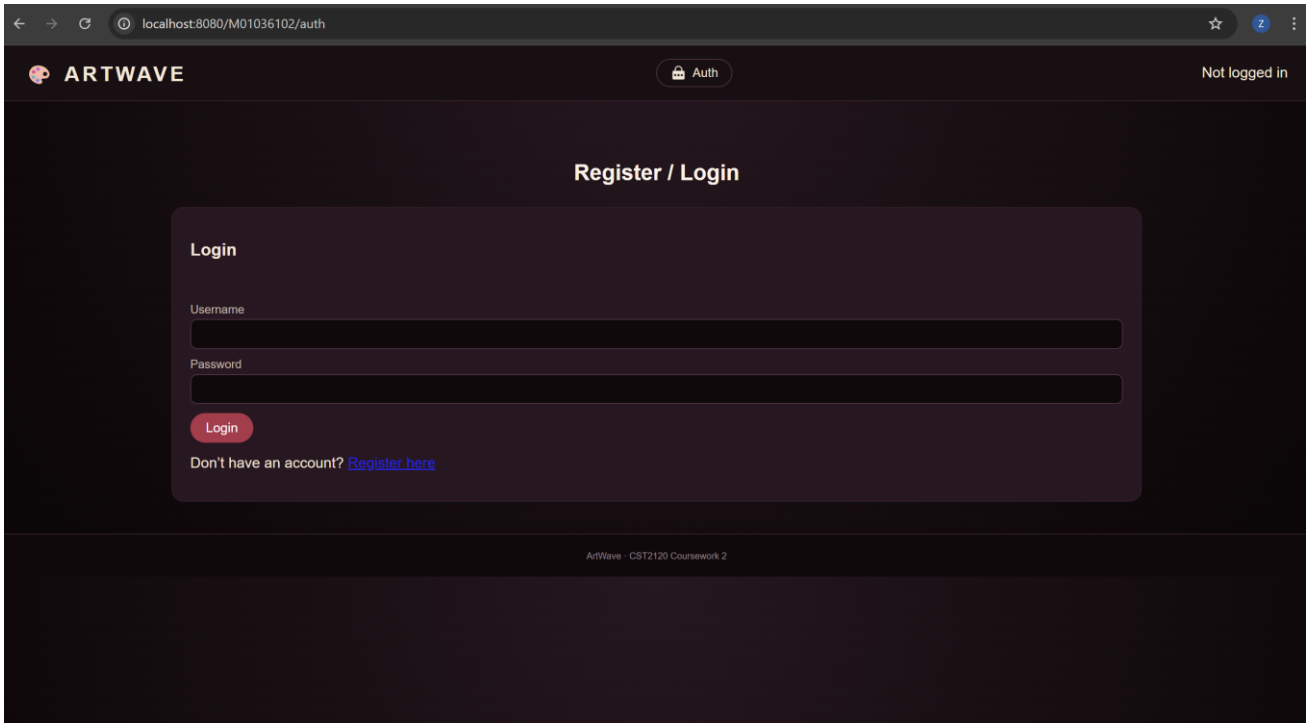


Fig. 2 Login Page (Home)

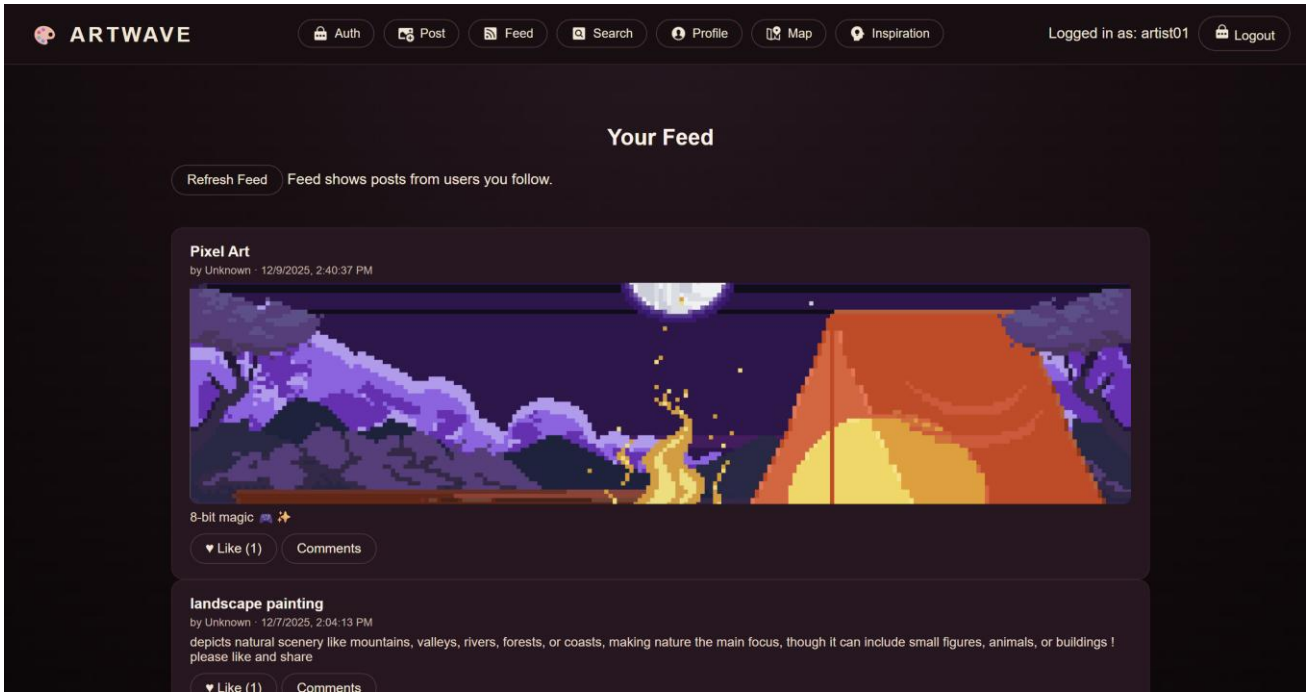


Fig. 3 Feed Page (User Logged in)

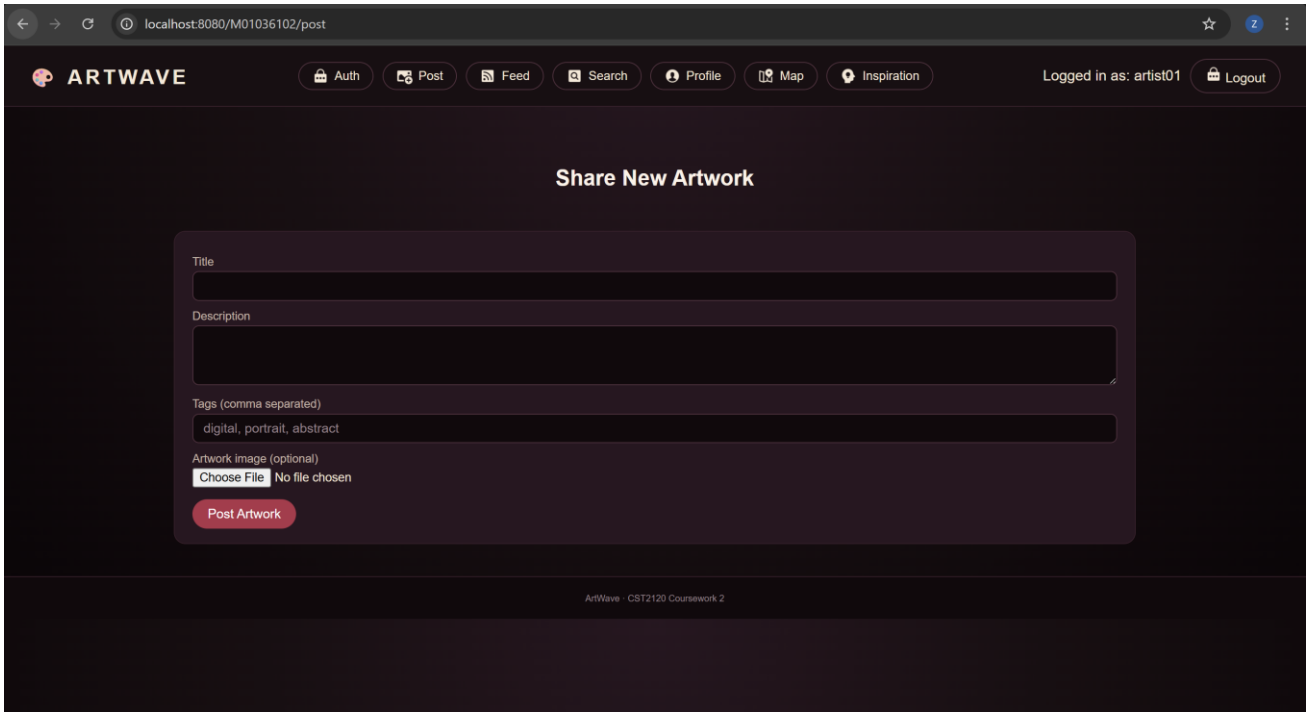


Fig 4. Create Art Posts Page

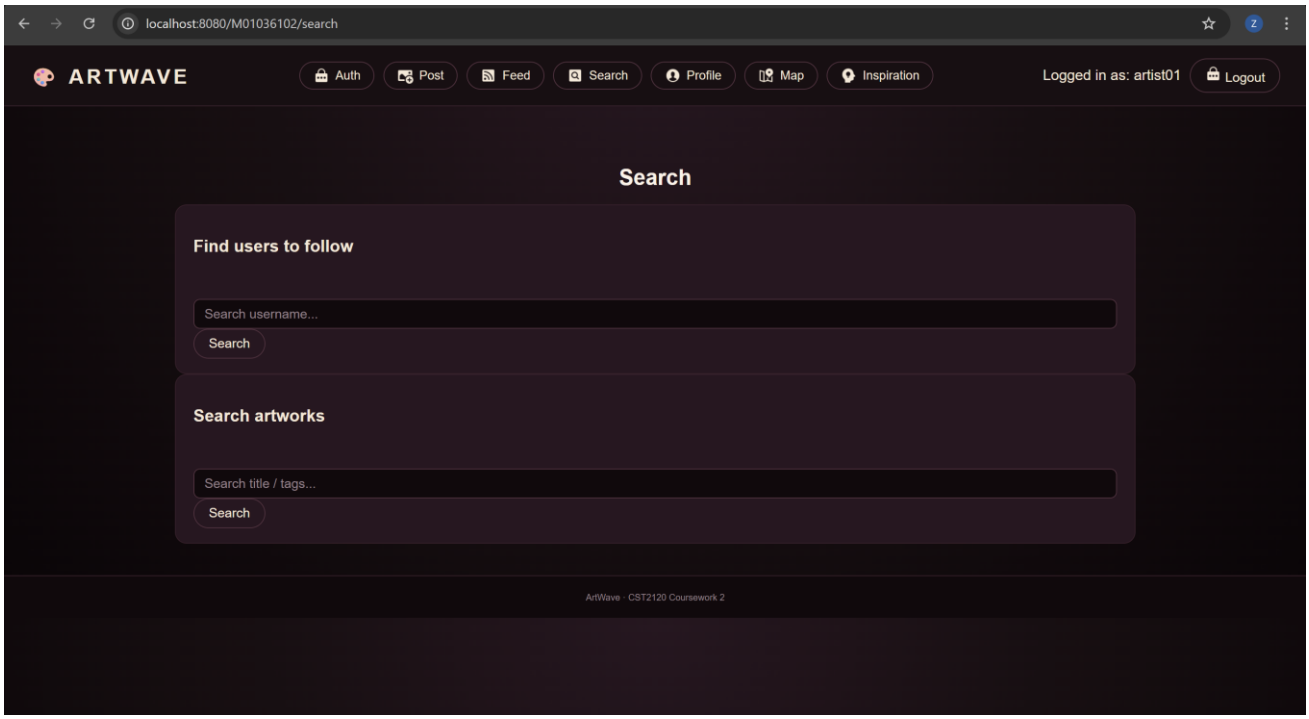


Fig 5. Search Page

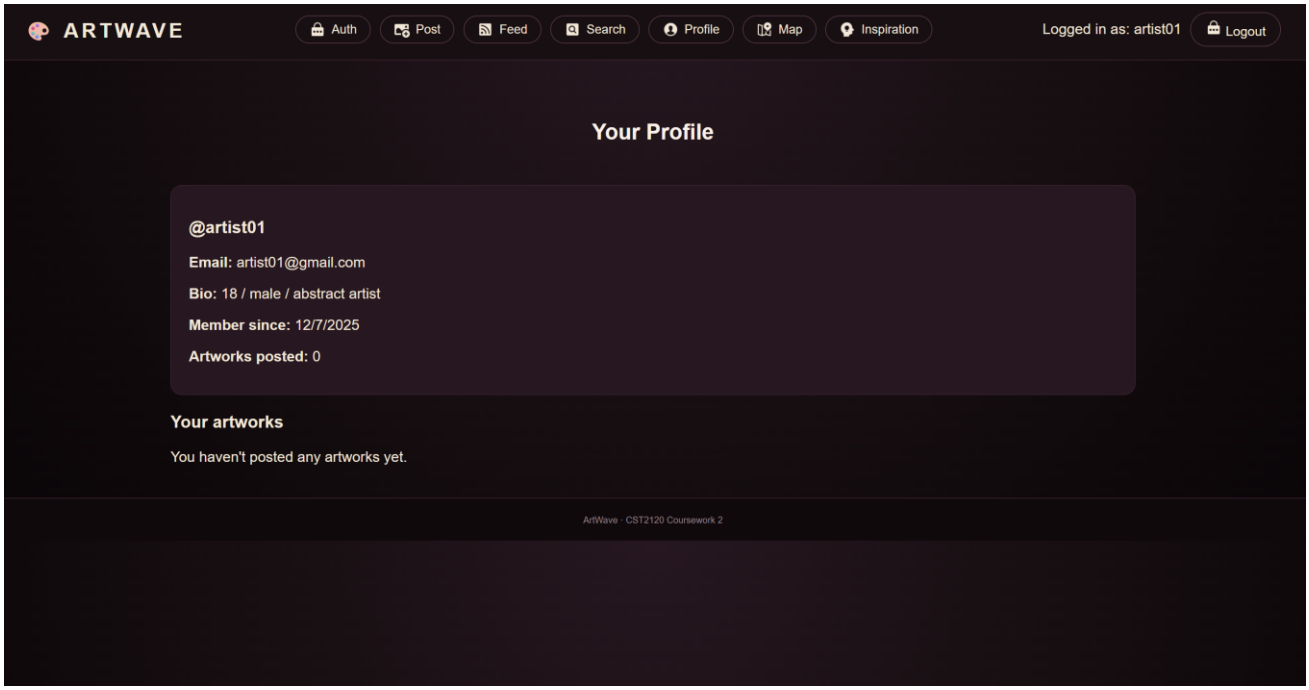


Fig 6. Profile Page

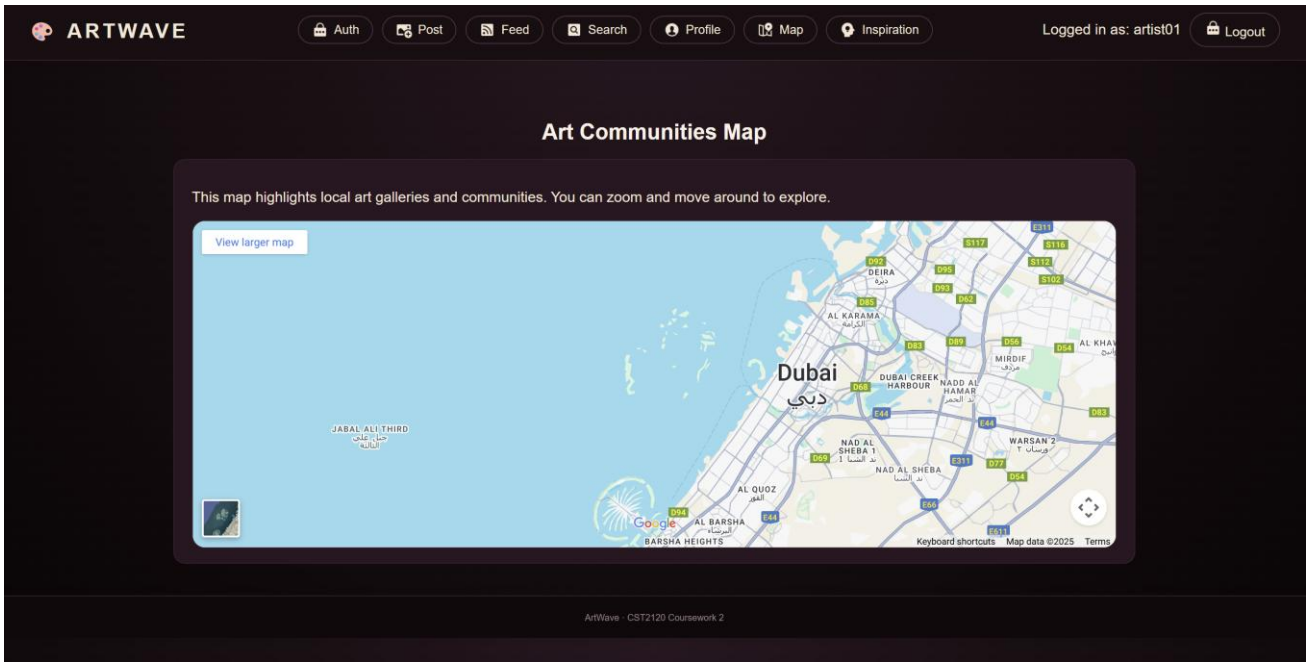


Fig. 7 Map Page

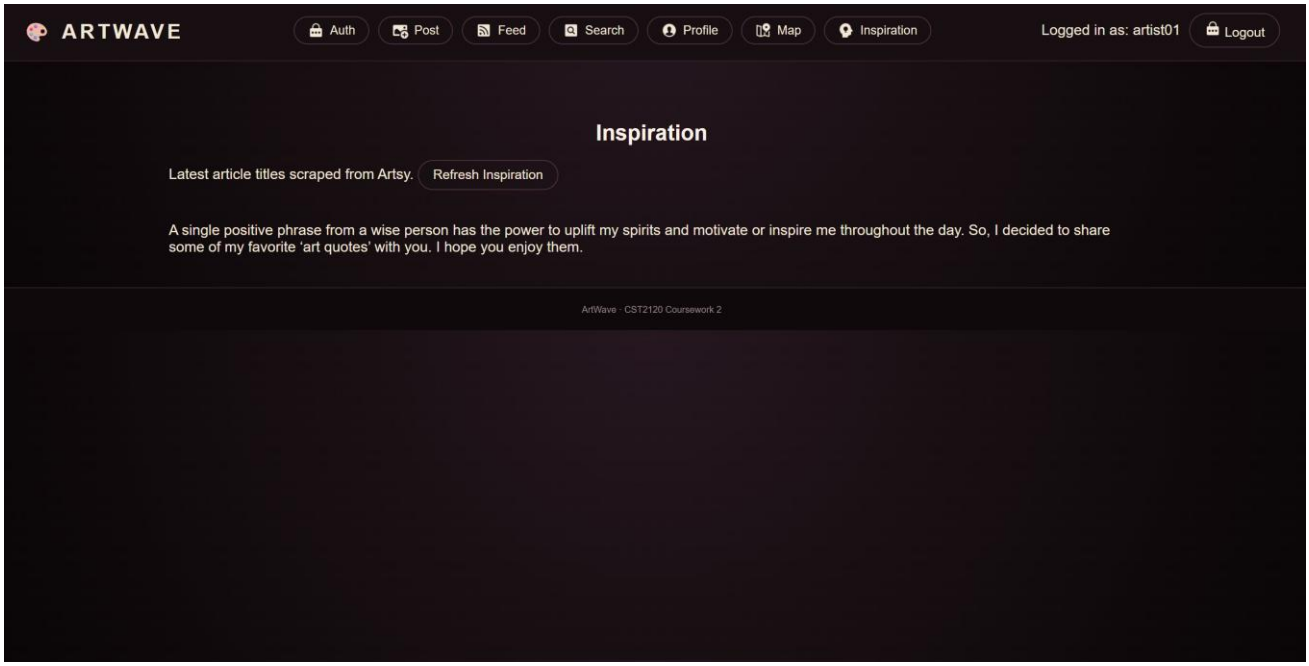


Fig 8. Inspiration Page (Quotes)

Back End of the Website (JavaScript & MongoDB)

The back-end of the ArtWave web app was put together with node.js and express.js. I chose them because it's pretty straightforward to get going with, and also very versatile: ideal for building all sorts of RESTful web applications. Express made it really easy to set up multiple API routes, each handling stuff like user login, posting artworks, following other users, liking content, and leaving comments, all while keeping the code super organized and modular.

MongoDB was used for the project's database, because it's got that document-based structure that fits a web app full of changing data like artworks, comments, and user interactions. The back-end deals with validating requests, checks who logged in, talking to the database, and sending over the right JSON responses to the frontend. Every route gets a prefix of the student ID slapped on the front as per the coursework brief.

Authentication was sorted using express-session. When a user finally gets logged in, we store their user id and username in the session, and then use that session to decide who gets to access the routes that are a bit locked down - posting art, viewing your feed, liking posts, and commenting for instance. That way we can make sure that only actual users get to use the social bits that most people care about.

.

Database Design

The database design of the ArtWave platform is structured across multiple mongodb collections to ensure clarity, and ease of querying. Each collection has a clear responsibility and represents a specific aspect of the system.

The users collection stores account information including usernames, email addresses, passwords, bios, and account creation dates. The artworks collection stores each artwork post along with metadata such as the author ID, title, description, tags, image path, upload date, and like count. Separating artworks into their own collection allows posts to be efficiently retrieved and displayed in feeds and profiles.

User relationships are handled using the follows collection, which stores follower and following user IDs. This enables the generation of a personalised feed based on followed users. The likes collection tracks which users have liked which artworks, allowing likes to be toggled without duplicating data. Finally, the comments collection stores comments associated with artworks, including the author, content, and timestamp.

Backend Testing and API Validation

To make sure the backend actually worked and right, I did a ton of testing with Postman to make sure all the key API endpoints were working. Testing each route on its own without having to mess with the frontend helped in tracking down any backend-related problems.

I tested user registration by firing off POST requests with some new user credentials and then checking to see if the account actually got stored in the database and for good measure, I also tested the login system by sending in both legit and bogus login credentials so I could see that the authentication was working the way it should. Plus I wanted to make sure that login status was staying consistent even when I made a bunch of test requests using cookies.

Also used Postman to test out the artwork creation, getting feed, likes, and comments on the site - all those things that only logged-in users should be able to do, and which should update the database as they go. And I checked out the error messages that I got back when I tried to send in bad requests, to make sure they were actually saying something useful.

Using Postman to do the backend testing basically gave me a whole lot of confidence in the system, and made it a lot easier to know that the frontend was talking to a backend that would actually work right.

Figure 1: User Registration (POST /users)

This screenshot shows a successful user registration request sent using Postman. The backend correctly validates the input data and creates a new user record in the database, returning a success message and user ID.

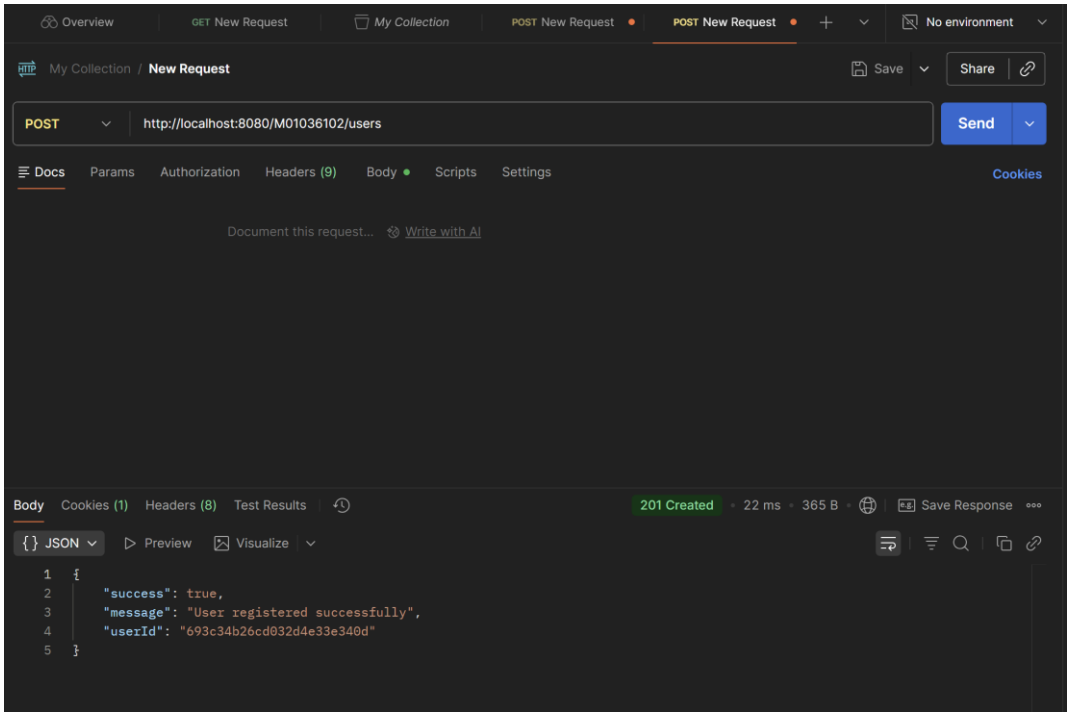


Figure 2: User Login (POST /login)

This figure demonstrates a successful login attempt. The server verifies the provided username and password, creates a session, and confirms authentication with a success response.

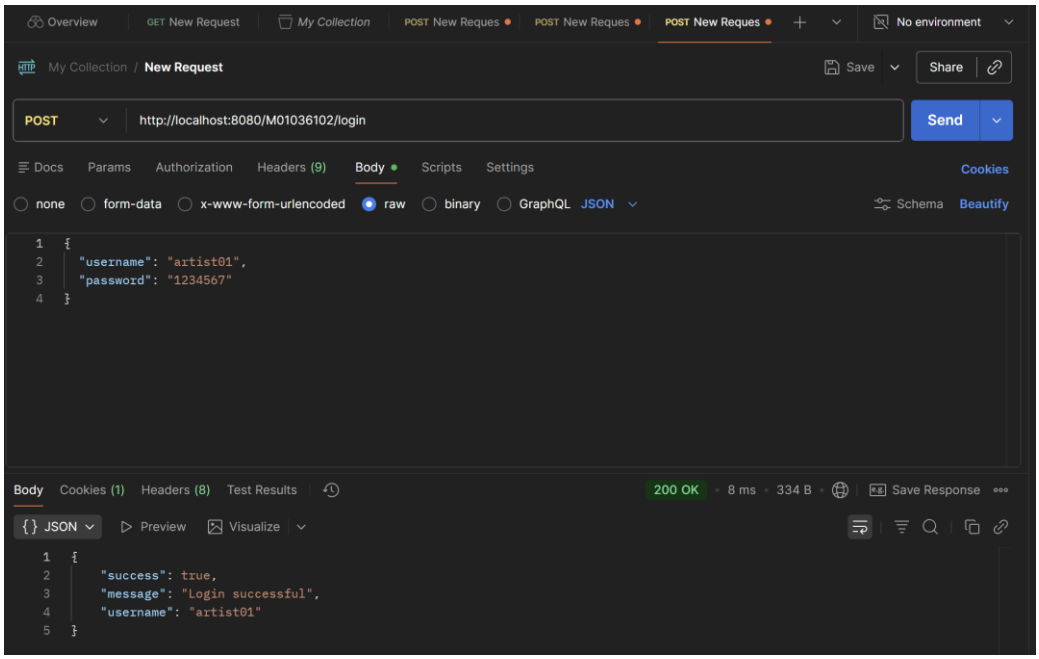


Figure 3: Login Status Check (GET /login)

This screenshot confirms that the user session is active after login. The backend correctly returns the logged-in status along with the authenticated user's information.

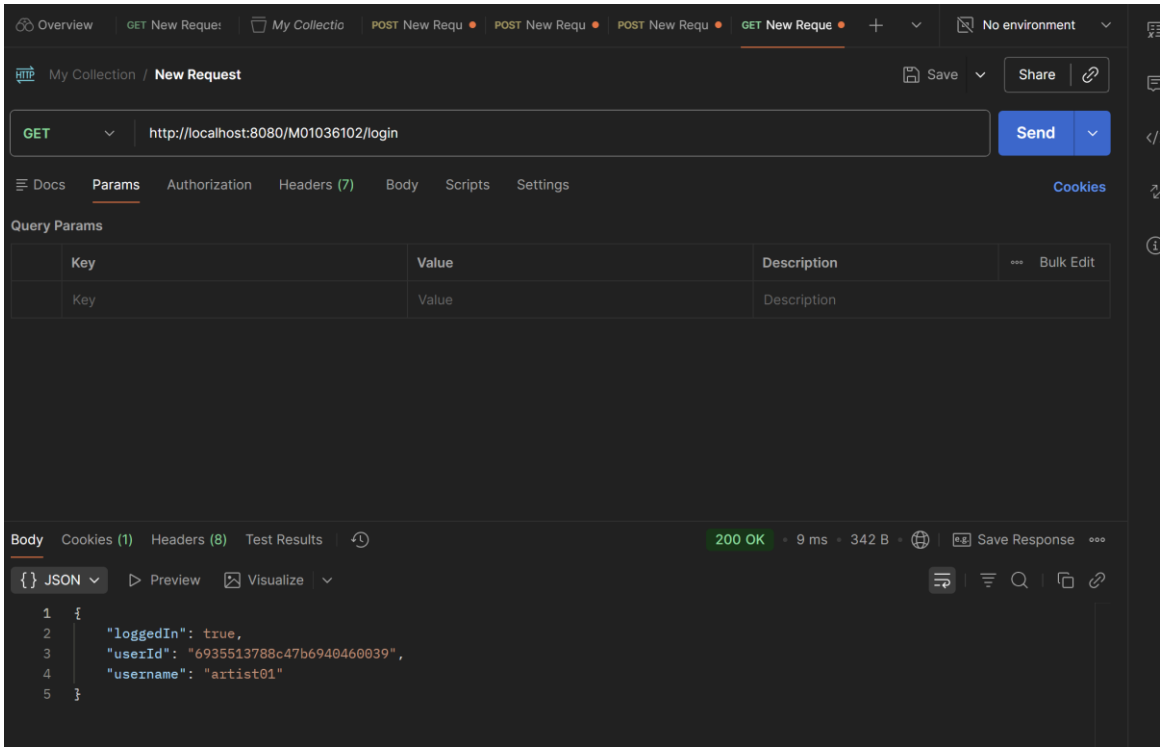


Figure 4: Create Artwork Post (POST /contents)

This figure shows the creation of a new artwork post. The backend validates the request, stores the artwork details in the database, and returns a unique artwork ID upon success.

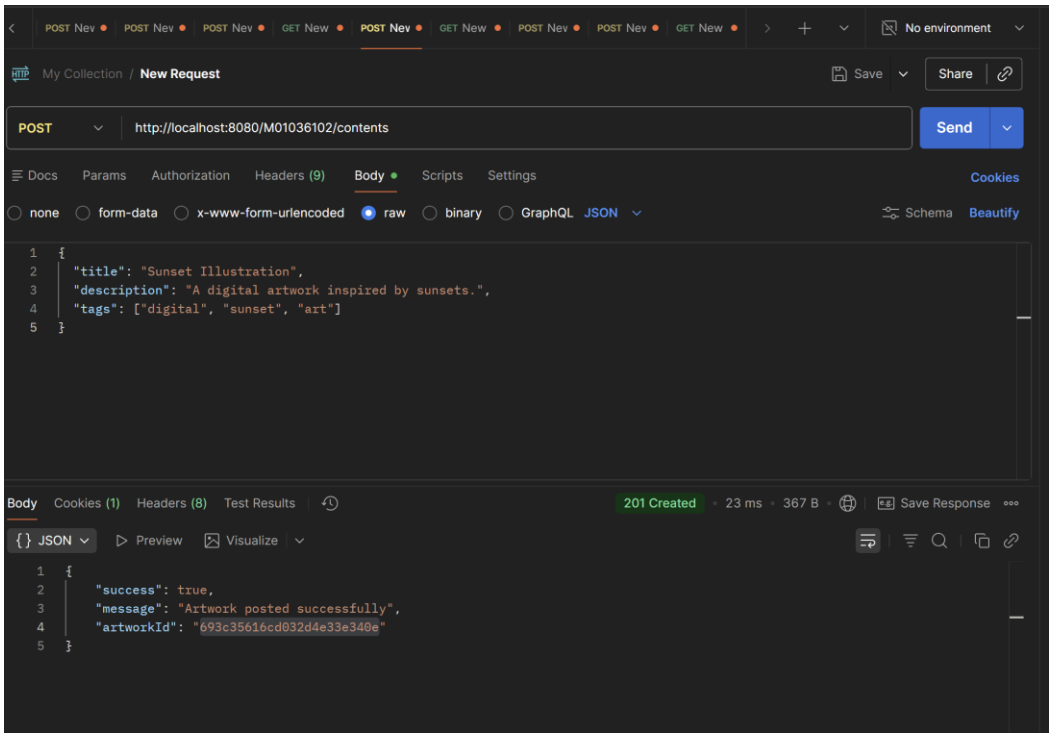


Figure 5: Retrieve User Feed (GET /feed)

This figure displays the personalised feed generated by the backend. The server retrieves artworks posted by users that the logged-in user follows and returns them in chronological order.

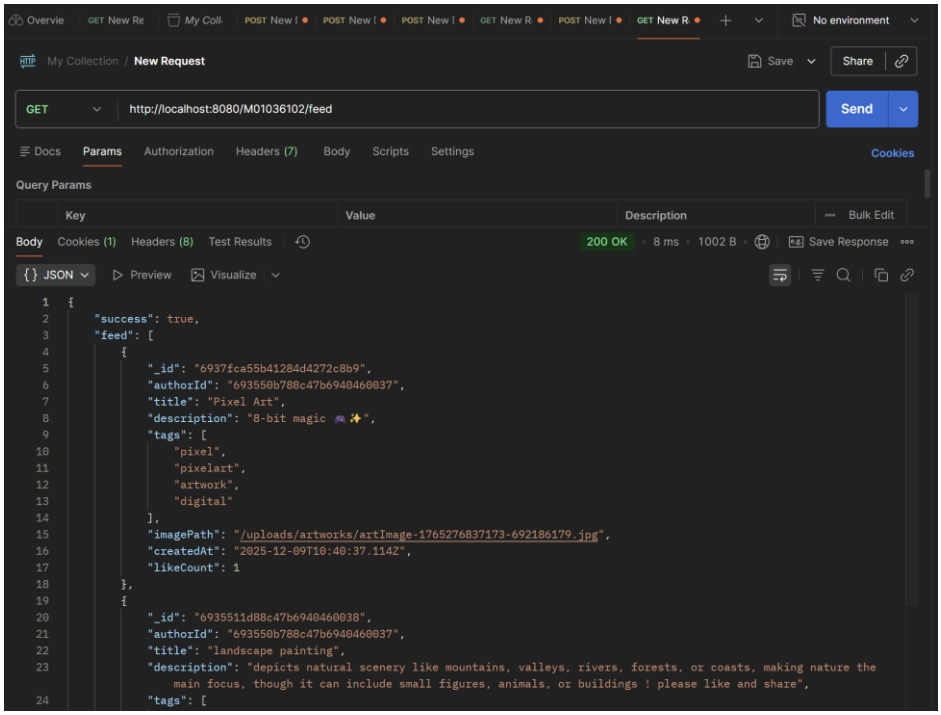


Figure 6: Like Artwork (POST /contents/{id}/like)

This screenshot shows the artwork like functionality in action. The backend records the like interaction, updates the artwork's like count, and returns the updated state.

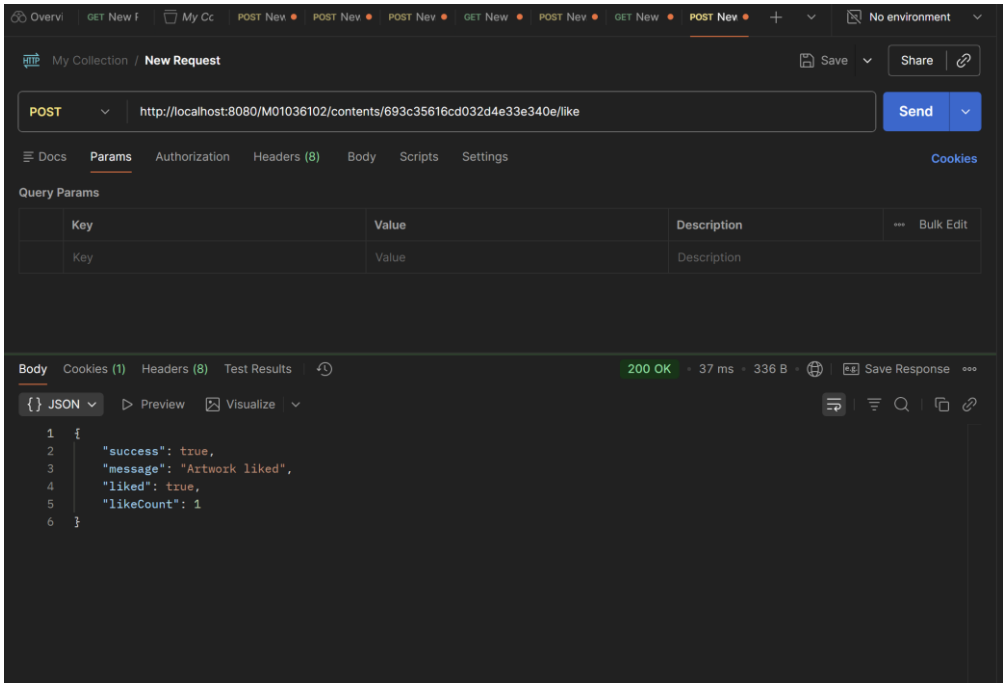


Figure 7: Add Comment to Artwork (POST /contents/{id}/comments)

This figure demonstrates adding a comment to an artwork. The backend validates the comment text, stores it in the database, and confirms that the comment was posted successfully.

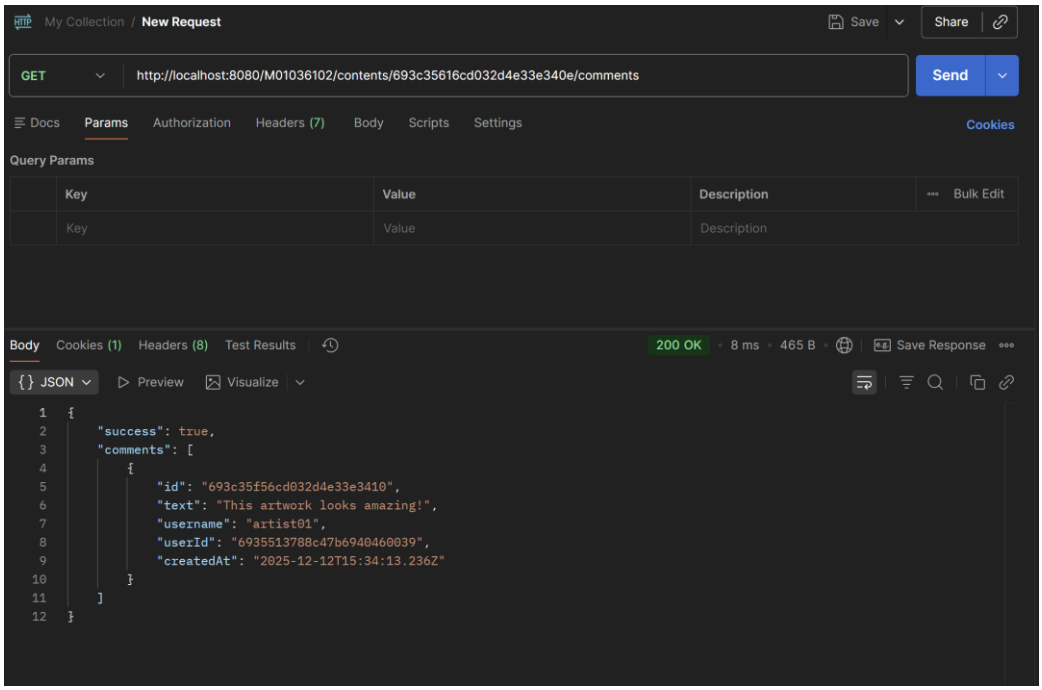


Fig 8: User Logout (DELETE)

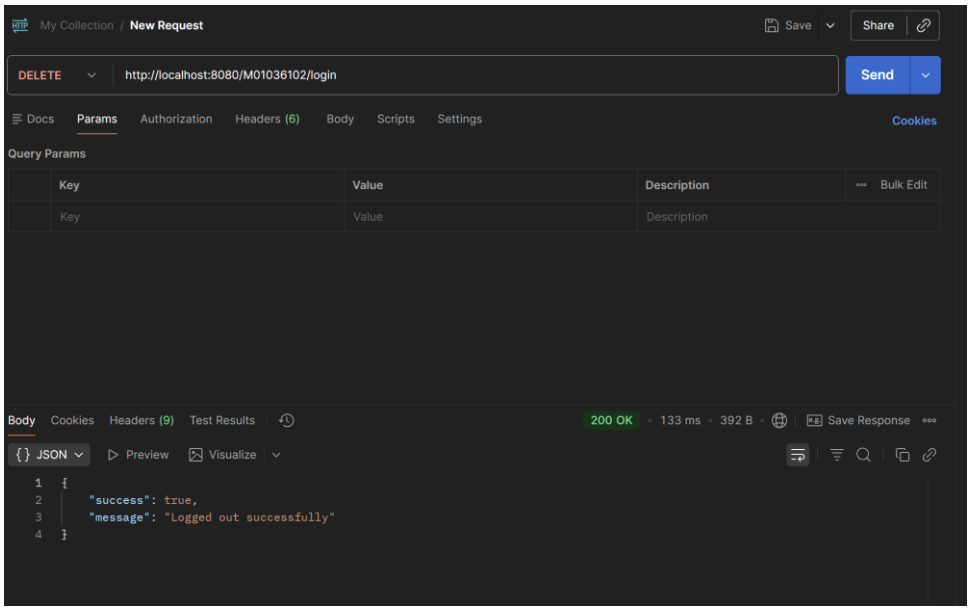


Fig 9: Searching for Artworks

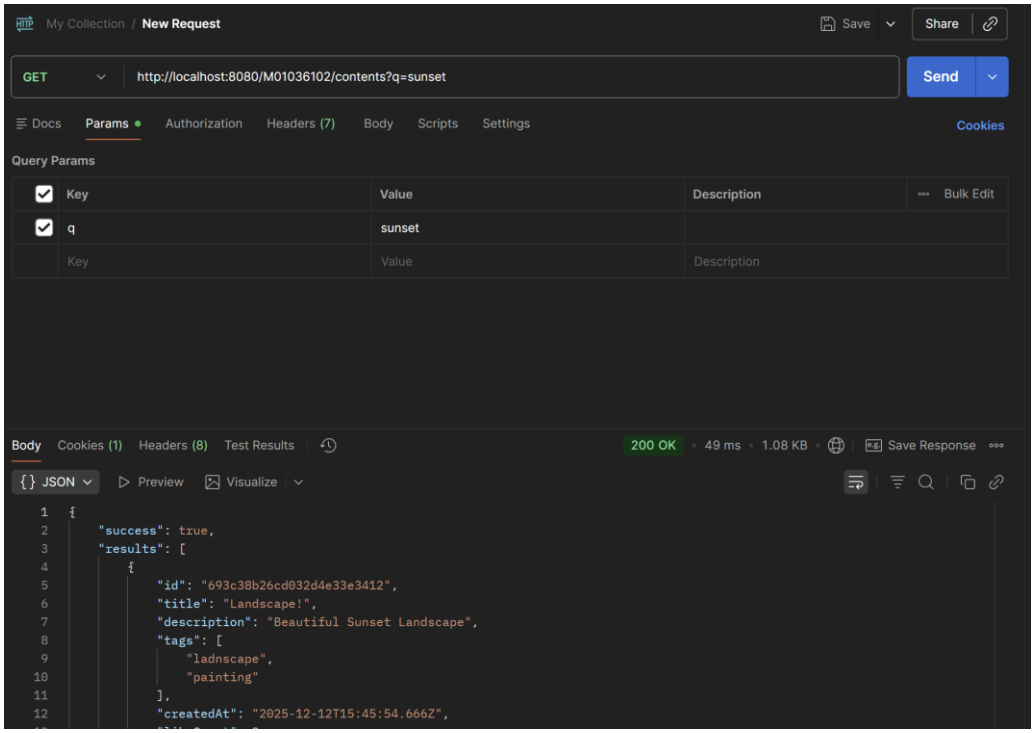


Fig 10: Get Comments (GET)

To see the comments under your posts

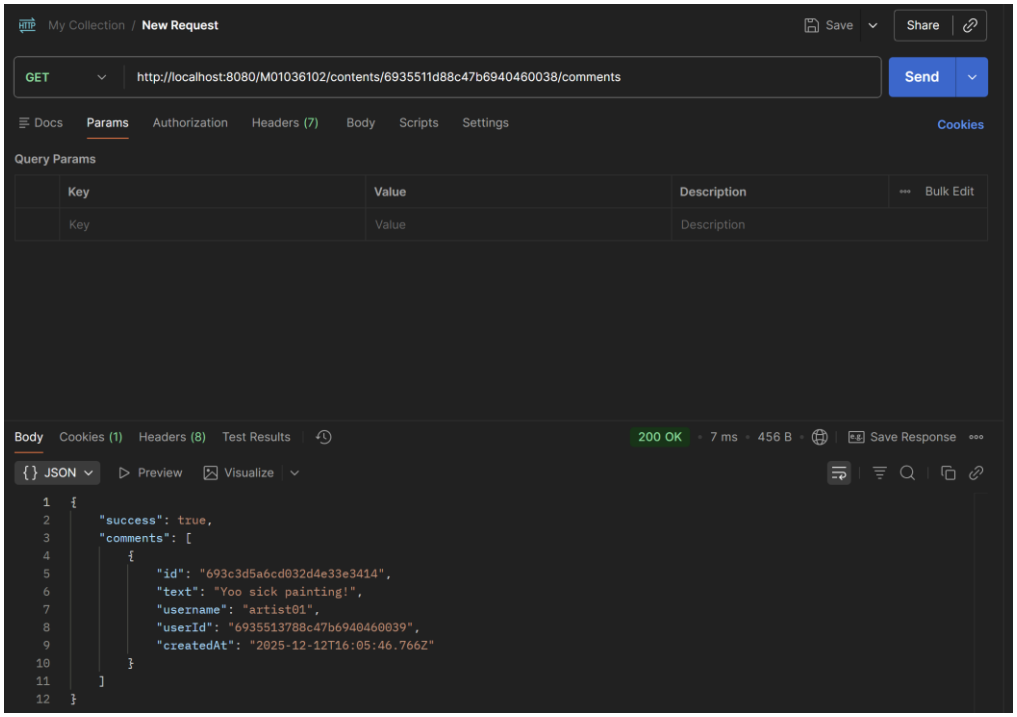


Fig 11: Search Users (GET)

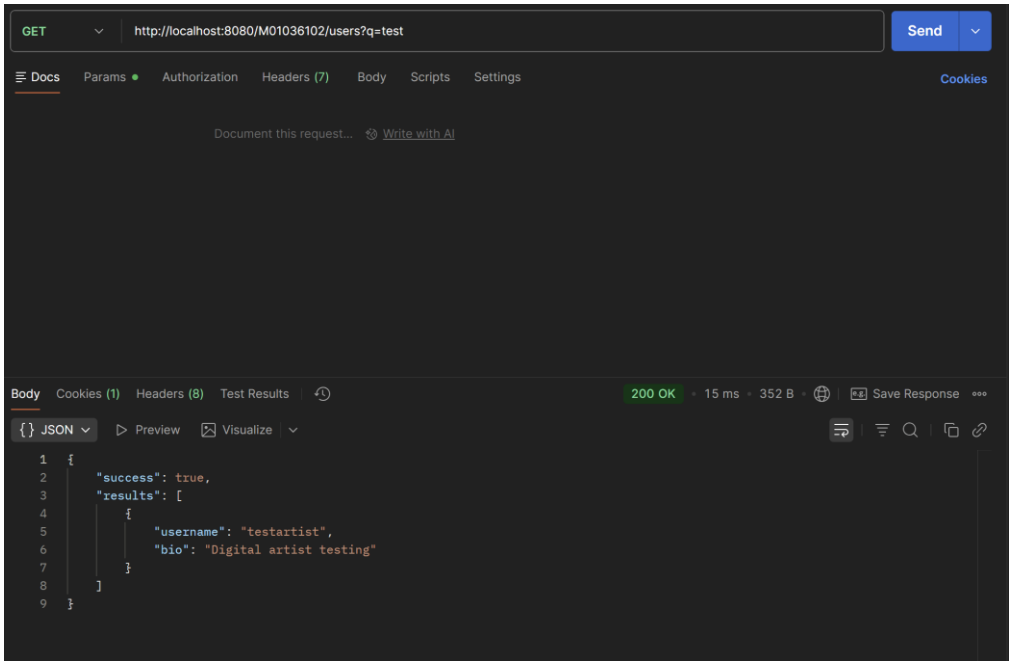


Fig 12: Follow Users (POST)

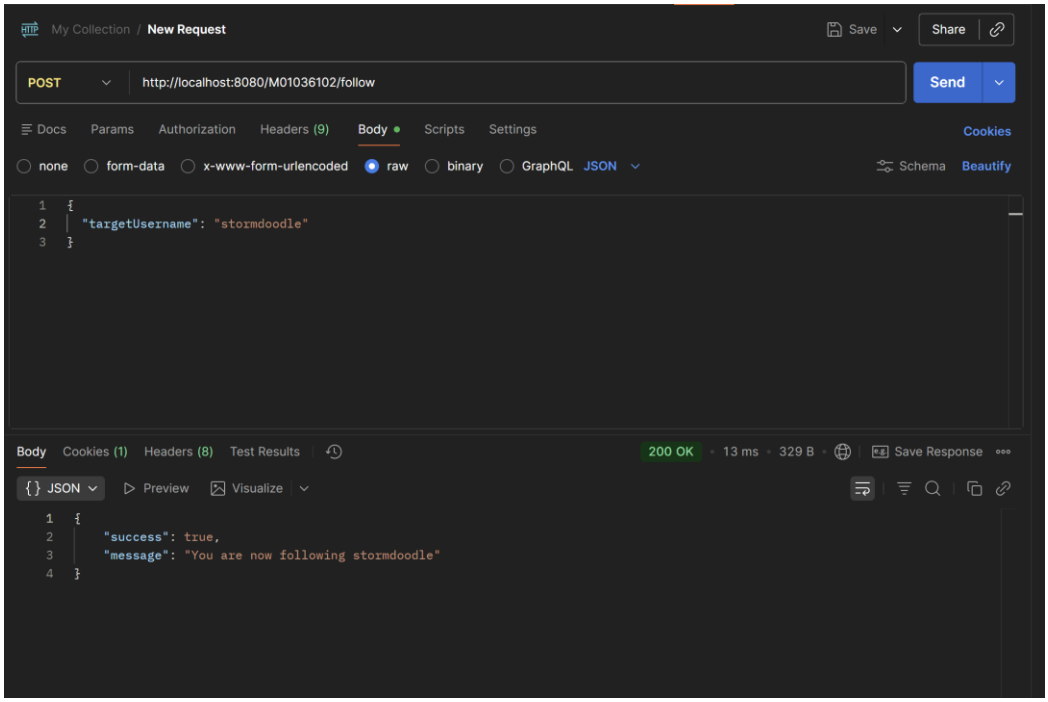


Fig 13: Unfollow Users (DELETE)

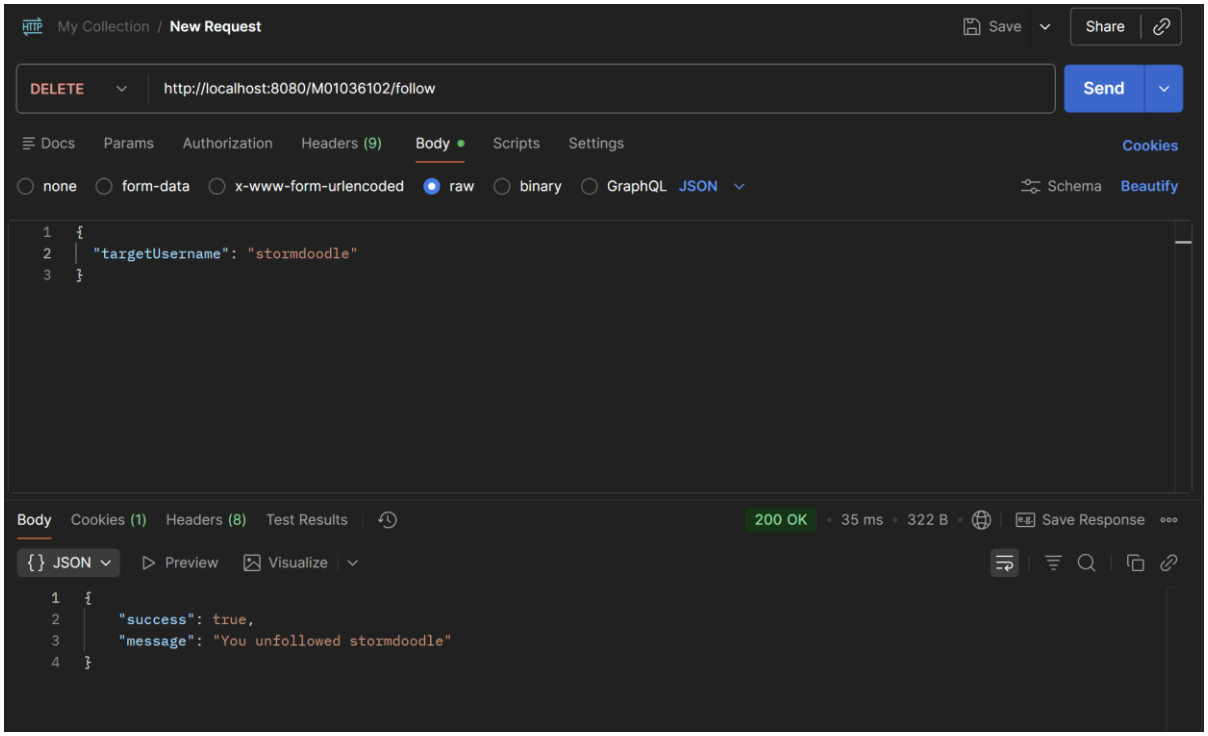
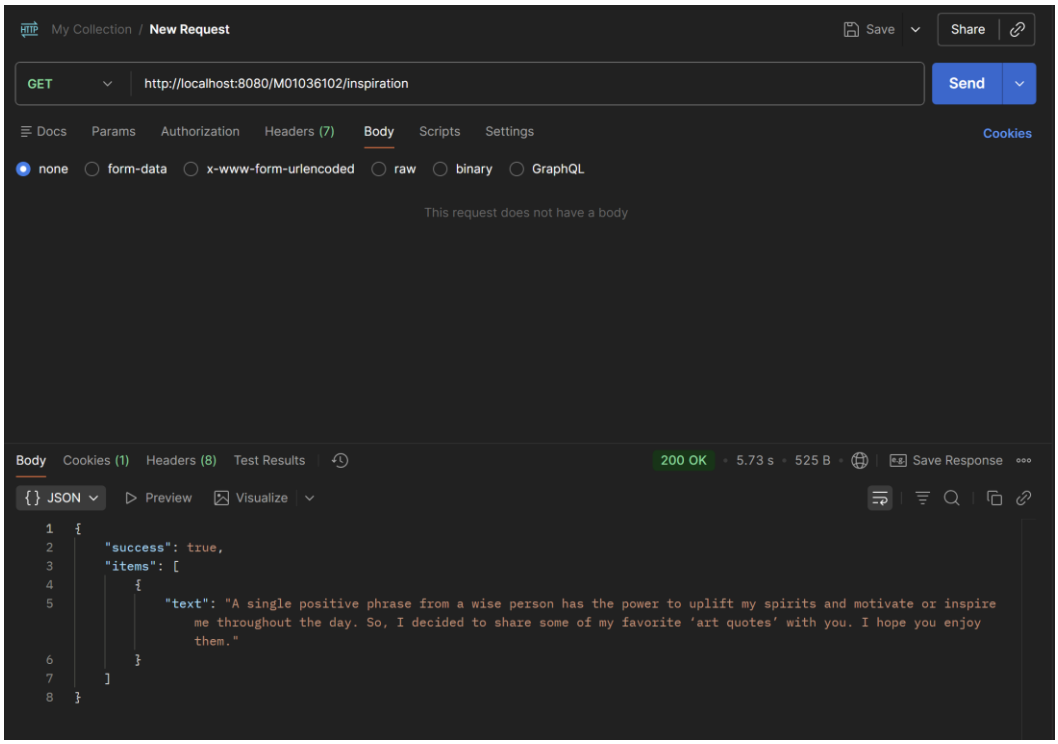


Fig 14: THIRD-PARTY / SCRAPING API

Art Inspiration (Web Scraping – Puppeteer)



Login, Registration and Authentication System

The ArtWave platform has got a complete login, registration & authentication system, that makes sure only users who have permission can get into and play around with the main parts of the website. When people join for the first time, they have to register by coming up with a unique username a valid email address and a password that'll be remembered. I check all their details on our server side before putting them away safely in our mongodb database.

Once you're all signed up , you can use your details to log back in. The way I handle logins on my backend makes use of sessions with express, which lets the server keep track of all the logged-in users in between requests and the best part is that once you've logged in successfully, a session gets created and your login status is safe and sound till you decide to log out, or the session finally runs out.

Any special features like posting art , seeing your own feed, following other users, liking posts and commenting won't be available unless we can see you've got a valid session going on.

If an unauthenticated user attempts to access these features, they are redirected back to the authentication section. This approach ensures proper access control while keeping the user experience simple.

Third-Party API and Web Scraping System

The ArtWave platform includes a third-party data feature to provide users with creative inspiration. This was implemented using Puppeteer, a Node.js library that enables server-side web scraping.

When the Inspiration section is accessed, the backend automatically visits a public art-related website and extracts relevant inspirational text. The scraped content is processed on the server and sent to the frontend in JSON format, where it is displayed dynamically without reloading the page.

This approach was chosen to demonstrate the ability to integrate external data sources when a dedicated public API is not available. Only publicly accessible content is retrieved, and basic error handling is implemented to ensure the system remains stable if the external source is unavailable.

This feature adds value to the platform while fulfilling coursework requirements for third-party data integration.

Evaluation and Reflection

Overall, the ArtWave project successfully met the core requirements of the coursework. The integration between frontend and backend was smooth, and the application supports user

authentication, content creation, social interaction, and dynamic data retrieval. The session-based authentication system worked reliably and prevented unauthorised access to protected features.

One challenge I facced during development was debugging session handling and ensuring that cookies were correctly maintained across requests. This required careful testing and validation using postman. Implementing the inspiration feature using web scraping was also challenging, as it required handling external website changes and potential failures.

If the project were to be extended further, several improvements could be made like for ex. hashing user passwords for better security, adding user profile pictures, and improving error handling and performance, despite these limitations, the project demonstrates a solid understanding of full-stack web development principles and backend system design.

Conclusion

In conclusion, the ArtWave project successfully shows the development of a full-stack web application that combines both technical functionality and user-centred design. platform allows users to register, log in, share artwork, interact with other users, and explore creative inspiration, fulfilling the main objectives set out in the project proposal.

The use of modern web technologies such as HTML, CSS, javascript, node.js, express, and mongodb enabled the creation of a responsive and interactive system. The separation between frontend and backend logic helped keep the application organised and easier to maintain. Features such as session-based authentication, image uploads, and feeds demonstrate a practical understanding of a web application development.

Even though the system has some limitations, such as basic password handling and less optimisation, it still meets the coursework requirements effectively. overall, this project provided valuable hands-on experience in full-stack development and reinforced key concepts taught throughout the module.

Usage of AI

Artificial Intelligence tools were used in a limited and supportive manner during the development of this project. AI assistance was primarily used to help clarify technical concepts, debug errors, and understand documentation related to javascript, node.js, and mongodb. AI helped me speed up problem-solving without replacing the actual implementation work.